



POLITECNICO
MILANO 1863

Progetto di Reti Logiche AA 2022-2023

Prof. Gianluca Palermo

Viganò Diego Valtolina Cristiano
Codice persona: 10739933 Codice persona: 10699057

Marzo 2023

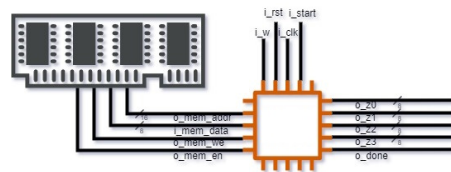
Indice

1	Introduzione	2
1.1	Interfaccia	2
2	Architettura	4
2.1	Gestione del segnale reset	6
3	Risultati Sperimentali	7
3.1	Velocità di risposta	7
3.2	TestBench	7
3.3	Constraints	8
4	Conclusioni	10
4.1	Possibili ottimizzazioni	10

1 Introduzione

L'obiettivo del progetto è di definire in VHDL un modulo hardware in grado di compiere le seguenti azioni:

- leggere l'ingresso seriale i_w in presenza del segnale i_start alto
- elaborarne il contenuto identificando una prima stringa di 2 bit rappresentante una delle quattro porte di uscita o_{z_x} del modulo e una seconda stringa di 16 bit rappresentante un indirizzo di memoria
- interrogare un componente di memoria utilizzando il suddetto indirizzo tramite il bus o_{mem_addr}
- esporre il contenuto dell'indirizzo di memoria sull'uscita o_{z_x} del componente individuata in precedenza
- altre e più dettagliate dinamiche di funzionamento del modulo sono descritte all'interno del file di specifica



1.1 Interfaccia

L'interfaccia del componente è definita come di seguito:

```
entity project_reti_logiche is
  port(
    i_clk  : in std_logic;
    i_rst  : in std_logic;
    i_start : in std_logic;
    i_w    : in std_logic;

    o_z0   : out std_logic_vector(7 downto 0);
    o_z1   : out std_logic_vector(7 downto 0);
    o_z2   : out std_logic_vector(7 downto 0);
    o_z3   : out std_logic_vector(7 downto 0);
    o_done : out std_logic;

    o_mem_addr : out std_logic_vector(15 downto 0);
    i_mem_data : in std_logic_vector(7 downto 0);
    o_mem_we   : out std_logic;
    o_mem_en   : out std_logic
  );
end project_reti_logiche;
```

Il preciso ruolo dei segnali di ingresso e uscita facenti parte dell'interfaccia è apprendibile dal file di specifica del progetto.

Al fine di memorizzare le due stringhe di bit rilevate dalla lettura dell'ingresso seriale, definiamo ulteriori due segnali interni al modulo:

```
signal s_mem_addr    : std_logic_vector(15 downto 0);  
signal z_id          : std_logic_vector(1  downto 0);
```

di cui il segnale *s_mem_addr* verrà riportato sull'uscita *o_mem_addr*

2 Architettura

A livello architetturale, la seguente implementazione del modulo hardware prevede unicamente l'uso di una macchina a stati finiti, qui denominati come "00", "01", "10" e "11". La scelta dell'architettura qui presentata è stata dettata dalla maggiore semplicità di gestione dell'ingresso seriale al modulo, la cui lettura e concatenamento vengono gestiti tramite semplici funzioni di cambio stato.

In seguito viene anche riportato uno *schema* del comportamento generale della macchina e relativa *tabella* dettagliata delle transizioni.

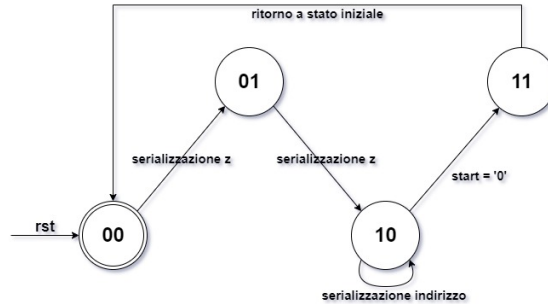


Figura 1: grafo rappresentante schematicamente la macchina a stati finiti

FSM - project_reti_logiche							
STATE	STATE_F	RST = 1		RST = 0			
//	//	START = X		START = 0		START = 1	
//	//	Next State	Funct.	Next State	Funct.	Next State	Funct.
00	$out_z_x \leftarrow 00_{hex}$ $o_mem_en \leftarrow '0'$ $o_done \leftarrow '0'$ $s_mem_addr \leftarrow 0000_{hex}$	00	$preout_z_x \leftarrow 00_{hex}$ $o_done \leftarrow '0'$ $out_z_x \leftarrow 00_{hex}$	//	//	01	z_id $\&$ i_w
01	$mem_en \leftarrow '1'$	00	$preout_z_x \leftarrow 00_{hex}$ $o_done \leftarrow '0'$ $out_z_x \leftarrow 00_{hex}$	//	//	10	z_id $\&$ i_w
10	//	00	$preout_z_x \leftarrow 00_{hex}$ $o_done \leftarrow '0'$ $out_z_x \leftarrow 00_{hex}$	11	//	10	s_mem_addr $\&$ i_w
11	$o_done \leftarrow '1'$ $out_z_x \leftarrow preout_z_x$ $out_z_id \leftarrow i_mem_data$	00	$preout_z_x \leftarrow 00_{hex}$ $o_done \leftarrow '0'$ $out_z_x \leftarrow 00_{hex}$	00	//	00	//

Le transizioni della macchina a stati finiti vengono realizzate esclusivamente in corrispondenza del fronte di salita del segnale di clock, eccezion fatta per tutti quei casi dove viene rilevato un se-

gnale di reset alto, nei quali le transizioni sono da considerarsi asincrone rispetto al segnale di clock.

Il primo stato della FSM, "00", funge anzitutto da stato di reset. La sua caratteristica funzione di stato permette infatti -come indicato in *tabella*- di compiere le seguenti operazioni:

00

- oscuramento delle uscite out_z_x del modulo hardware, alle quali viene assegnato il valore 00_{hex}
- segnale o_done posto a 0
- segnale o_mem_en per la lettura in memoria posto a 0
- al segnale rappresentante l'indirizzo di memoria da costruire viene assegnato il valore iniziale 0000_{hex}

La transizione da questo stato verso "01", permette di costruire il primo bit del vettore z_id , che identifica l'uscita sulla quale verrà esposto il segnale dati in uscita dal componente di memoria.

01

Lo stato "01" è principalmente utile alla lettura del secondo ed ultimo bit rappresentante l'indirizzo di uscita z_id , implementata durante la transizione verso lo stato prossimo "10".

Un segnale i_start che abbia una durata maggiore o uguale a due cicli di clock, come garantito da specifica, obbliga quindi la percorrenza di queste due prime transizioni, assicurando la corretta costruzione dell'identificatore della porta di uscita. La funzione di stato di "01" permette di:

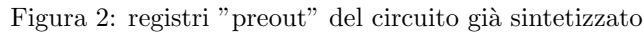
- porre a '1' il segnale di o_mem_en per la lettura in memoria. Ciò permette, nel caso di abbassamento del segnale i_start , di leggere da memoria il dato corrispondente all'indirizzo 0000_{hex} , impostato come valore di default dalla funzione di stato di "00".

Lo stato "10" presenta un comportamento ciclico. Ogni sua transizione riflessiva permette la lettura e il concatenamento dell'ingresso seriale i_w . La costruzione dell'intero indirizzo di memoria sarà perciò totalmente delegata a questo stato. Il segnale o_mem_en di enable alla lettura in memoria viene mantenuto alto. La transizione allo stato "11" avviene in maniera sincrona al segnale di clock e solamente come conseguenza della rilevazione di un segnale i_start basso.

10

11

Lo stato "11" ha come unico scopo quello di esporre sulle uscite del modulo hardware " out_z_x " i dati letti da input seriale i_w . Parallelamente, il segnale di uscita o_done sarà posto ad '1', indicando la corretta elaborazione del segnale ricevuto. Al fine di poter mostrare in uscita, coerentemente alla specifica, anche i risultati delle letture precedenti, sono stati introdotti quattro *registri* di memoria, denominati " $preout_z_x$ ", utilizzati per memorizzare volta per volta i vettori di dati in uscita dal modulo, sovrascrivendoli se necessario.



Lo stato prossimo di "11" è unicamente "00". Il comportamento della macchina qui descritto permette perciò di mantenere le uscite visibili per un solo ciclo di clock, per poi oscurarle non appena lo stato corrente commuta a quello di reset.

Non è inoltre sufficiente sfruttare le proprietà dello stato "00" per riportare la macchina in uno stato di totale azzeramento; ciò è dovuto alla presenza dei registri "preout" utilizzati per la memorizzazione successiva delle uscite. Qualsiasi transizione da uno stato "XX" allo stato "00", dovuta alla ricezione di un segnale di reset, dovrà quindi assicurarsi che i segnali di *preout* vengano reinizializzati al valore 00_{hex}.

3 Risultati Sperimentali

3.1 Velocità di risposta

Il modulo hardware così implementato è in grado di produrre un output coerente con le sue specifiche di funzionamento nel tempo di 1.5 cicli di clock. La seguente *immagine* mostra, il caso particolare di lettura in memoria all'indirizzo 0000_{hex} .

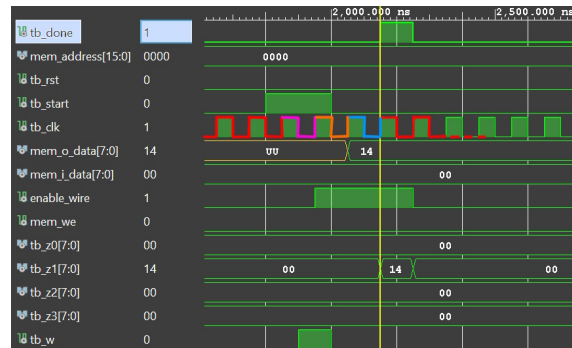


Figura 3: Rappresentazione delle transizioni di stato in fase di lettura

Il segnale di clock è stato evidenziato al fine di fornire una lettura più immediata dello stato nel quale si trova la macchina durante l'elaborazione del segnale seriale.

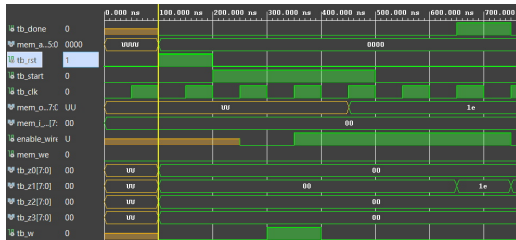
- **Stato 00** → ROSSO → lettura del primo bit di Z → transizione verso "01"
- **Stato 01** → VIOLA → lettura del secondo bit di Z → transizione verso "10"
- **Stato 10** → ARANCIONE → rilevazione del segnale *i_start* basso → transizione verso "11"
- **Stato 11** → BLU → esposizione dei valori in uscita e commutazione del segnale *o_done* a "1" → transizione verso "00"

3.2 TestBench

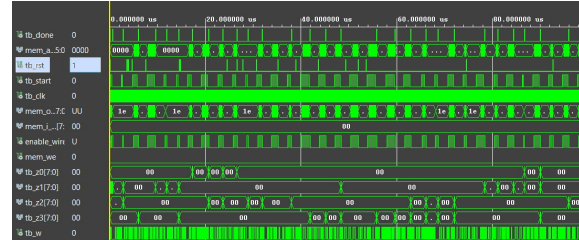
Il corretto funzionamento comportamentale del modulo qui descritto è stato analizzato utilizzando, in aggiunta ai testbench forniti assieme alla specifica, dei casi di test contenenti condizioni limite o singolari a cui potrebbe essere sottoposto il modulo.

- **Segnale di reset asincrono** Come già anticipato nella sezione *Architettura*, il segnale di reset è stato considerato asincrono rispetto al segnale di clock. Ciò implica che la ricezione del segnale di reset e la sua conseguente gestione avvenga indipendentemente dalla presenza di un fronte di salita sul segnale di clock.
- **Segnale di reset di minima durata** La ricezione del segnale di reset è da considerarsi valida anche se mantenuta per un solo ciclo di clock.
- **Segnale di start immediatamente successivo al fronte di discesa del segnale di reset** Non appena terminato il segnale di reset, il modulo hardware deve essere immediatamente pronto a rispondere ad un segnale di inizio sequenza.

- **Scenario di funzionamento continuativo** E' stato ritenuto adeguato analizzare il comportamento del modulo hardware a fronte di scenari di elaborazione continuati nel tempo. La seguente immagine mostra infatti il corretto funzinnamento del modulo a fronte di uno scenario della durata di 100 μ s, pari a 1000 cicli di clock con periodo di 100ns



(a) presenza di reset asincrono rispetto al clock, di minima durata ed immediatamente precedente ad un fronte di salita del segnale START



(b) corretta esecuzione di una sequenza di elaborazione della durata di 100 μ s

3.3 Constraints

Al fine di verificare il corretto funzionamento del modulo hardware in fase di post-sintesi, è stato inserito un vincolo di forma d'onda del segnale di clock.

```
create_clock -period 100.000 -name clk -waveform {50.000 100.000} [get_ports i_clk]
```

Figura 5: codice rappresentate il vincolo imposto al segnale di clock

Segue quindi l'analisi statica delle prestazioni del modulo successivamente alla sintetizzazione circuitale.

Design Timing Summary			
Setup	Hold	Pulse Width	
Worst Negative Slack (WNS): 97,536 ns	Worst Hold Slack (WHS): 0,139 ns	Worst Pulse Width Slack (WPWS): 49,500 ns	
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns	
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	
Total Number of Endpoints: 174	Total Number of Endpoints: 174	Total Number of Endpoints: 89	
All user specified timing constraints are met.			

Figura 6: tabella di riepilogo temporale della simulazione statica del modulo

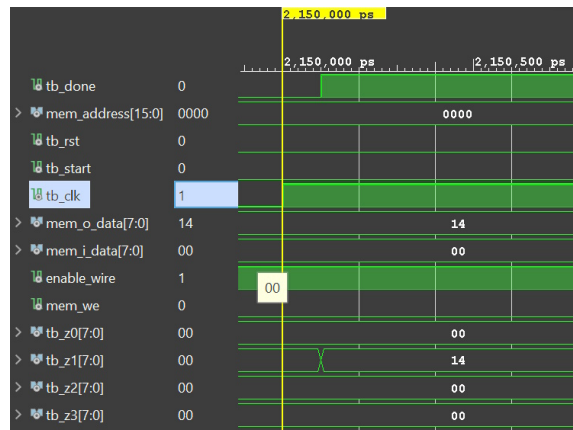


Figura 7: zoom-in sul ritardo di commutazione dei segnali, non più perfettamente sincrono al fronte di salita del clock

I ritardi temporali considerati durante questa fase di test del modulo circuitale non hanno in alcun modo compromesso il corretto funzionamento di quest'ultimo, che si mantiene pienamente nei limiti previsti dalla specifica.

4 Conclusioni

4.1 Possibili ottimizzazioni

L' *architettura* corrente è stata inizialmente pensata per essere in grado di esporre in output i risultati derivati dalla lettura della porta seriale, in corrispondenza del fronte di salita immediatamente successivo al fronte di discesa del segnale start, riducendo il tempo di risposta rispetto all'implementazione presente di un ciclo di clock.

Il comportamento qui descritto è ottenibile collassando lo stato "11" nello stato "10", e mantenendo sempre attivo il segnale di enable alla memoria.

La non idealità nella risposta della memoria tuttavia impedisce una corretta commutazione del segnale in uscita.

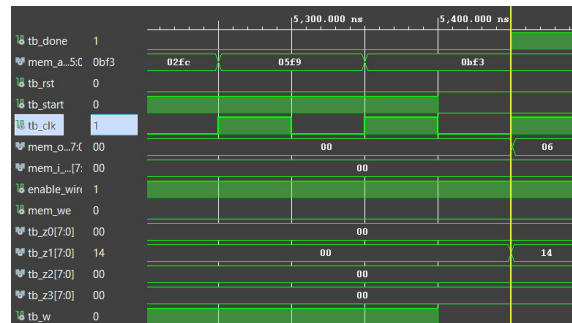


Figura 8: impossibilità di commutazione del segnale a causa del ritardo di 1ns sui dati in uscita dalla memoria

La qui descritta problematica di funzionamento modulo hardware può essere risolta nelle seguenti modalità:

- **Aggiunta di un ritardo fisso** prima dell'esposizione dei risultati in output: un ritardo maggiore o uguale a 1ns, in questo caso, permetterebbe la corretta commutazione del segnale dati in uscita dalla memoria prima di essere copiato in output al componente.
- **Utilizzare una funzione di controllo sulla commutazione dell'output proveniente dalla memoria** permetterebbe di avviare la commutazione dei segnali in uscita dal modulo hardware esclusivamente a seguito della corretta ricezione dei dati dalla memoria.

Le precedenti ottimizzazioni non sono tuttavia state implementate nella versione finale del progetto a causa delle eccessive complicazioni introdotte durante le fasi di analisi temporale post-sintesi. Si è preferito inoltre non introdurre delay temporali fissi.

La presente implementazione è stata altresì scartata poiché si è preferito -data la semplicità di funzionamento del modulo hardware- optare per un'architettura maggiormente leggibile.